R_GSoC2026_Proposal

March 31, 2026

0.1 Project Info

Project title: hyperSpec-HPC: Fast Sparse Spectral Kernels in R and Rust

Project short title (30 characters): Fast graph-based smoothing

URL of project idea page: github.com/rstats-gsoc/gsoc2026/wiki/hyperSpec-HPC

0.2 Bio

I am Tamaghna Chattopadhyay, a pre-final year student pursuing a B.Tech in Electronics Engineering at IIT(BHU) Varanasi. I have a deep interest in Systems programming and Mathematics. I like to study and understand the working and complexities of different algorithms. As a Electronics student, I have learnt about spectroscopy and filtering through my coursework. These subjects sparked my interest because of the rich mathematical foundations behind them and their practical applications in analyzing real-world signals. I have worked on various lab projects involving filtering and image processing using Python. Additionally, I am an upcoming Systems Intern at Nvidia and have worked as a embedded systems engineer at a startup before.

I was first introduced to R through a university workshop where we were taught the basics and given some tasks. Later, I explored R and its packages on my own and used it on some personal work related to data analysis. I have learnt Rust programming by myself through code crafters and the Rust Book (yet to fully finish), and worked on a Rust CLI from scratch project.

I had contributed to some open source projects, most notable contributions in Jenkins and Meshery, with around 20+ merged PRs. Through these contributions, I had learned to work and be comfortable with large codebases. Last year, I applied to Jenkins for GSoC with a proposal focused on refactoring and optimizing the backend of an infrastructure tool. Unfortunately, the project was not shortlisted due to a lack of mentors, and I was not selected. However, the experience motivated me to improve my skills and continue to work on myself.

0.3 Contact Information

Contributor name: Tamaghna Chattopadhyay

Contributor postal address: 64 N.S. Avenue, Serampore, Hooghly, West Bengal, India, 712201.

Telephone(s): +91-9674889722

Email(s): chattopadhyaytamaghna@gmail.com, tamaghna.c.ece23@itbhu.ac.in

Other communications channels:
Hangouts: chattopadhyaytamaghna@gmail.com

Discord: tommichan101

0.4 Contributor Affiliation

Institution: Indian Institute of Technology (BHU), Varanasi, India

Program: B.Tech. in Electronics Engineering

Stage of completion: Third Year, expected completion in 2027

Contact to verify: Dr. Amritanshu Pandey, head.ece@iitbhu.ac.in

0.5 Schedule Conflicts

I do not have any major conflicts that will affect the project. I have a summer break of more than 2 months and will be able to dedicate 40+ hours/week to R GSoC. The first week of May will be a bit tight as I will be having my end semester exams, but I have planned ahead to compensate for that.

0.6 Mentors

Evaluating mentor name and email: Erick Oduniyi (eeoduniyi@gmail.com)

Co-mentor name(s) and email(s):

I have been in touch with Erick Oduniyi through Email, starting March, 2, 2026.

1 Coding Plan and Methods

1.1 Project Background

The hyperSpec package provides a suite of tools for working with spectroscopic data in R and is widely used for analyzing hyperspectral datasets such as Raman and NIR imaging. While the package has evolved significantly over the years, undergoing fortification in GSoC 2020, the computational backend remains a bottleneck for large datasets. Many operations such as spectral filtering, baseline correction, and dimensionality reduction are currently implemented in pure R or rely on dense matrix computations, which do not scale efficiently for large hyperspectral images. Hyperspectral images naturally contain spatial structure, where neighboring pixels often share similar chemical properties. This makes them well suited for graph-based processing, where pixels are treated as nodes and spatial adjacency defines edges. Using the graph Laplacian, spatial smoothing can reduce noise while preserving spectral feature.

Hyperspectral datasets can be extremely large (e.g., 512 x 512 pixels x 2048 wavelengths, roughly around 4 GB), and thus pure R implementations begin to struggle. This project proposes implementing a Rust-powered backend for sparse Laplacian spatial smoothing. By using efficient sparse solvers with O(N) complexity, the Rust implementation can significantly outperform the pure-R approaches.

1.2 Project Goal

The goal of this project is to implement a high-performance Rust backend for graph-based spatial smoothing of hyperspectral images. This will include creation of a new `hyperspec.rust` R package

via extendr with CI and basic tests, and implementing a reliable bridge between R's `dgCMatrix` sparse matrix format and Rust's `faer` sparse matrix representation. The Rust side will construct pixel neighborhood graphs using `petgraph` and implement a sparse Laplacian solver using `faer` iterative solvers (CG/BiCGSTAB) to compute the smoothing operation $(I + \alpha L)x = b$.

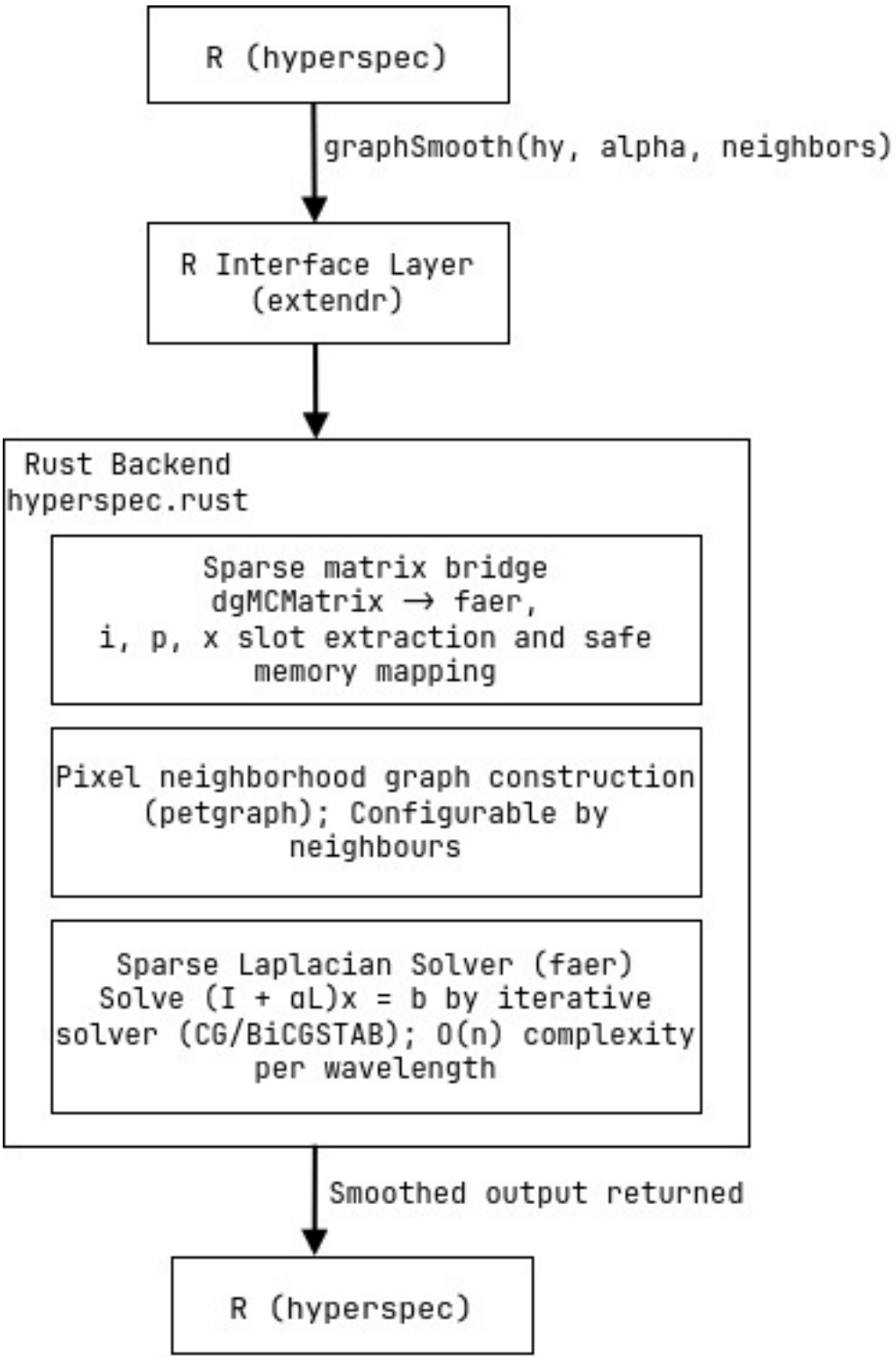
An R-level S4 method **graphSmooth(hy, alpha, neighbors)** will expose the functionality to users. The Rust implementation will generate adjacency graphs based on configurable neighborhood connectivity (e.g., 4 or 8-connectivity) to minimize data transfer from R.

The project will also include performance benchmarks to compare the Rust kernel with pure-R implementation for large hyperspectral datasets (e.g., 512×512×2048, ~4GB) and a vignette to demonstrate spatial smoothing of hyperspectral Raman maps using graph filtering.

Some extra goals that we plan to complete:
i. Publishing the hyperSpec-HPC package to CRAN
ii. Publishing the dgCMatrix → faer FFI bridge pattern as Rust crate
iii. Publishing a JOSS paper documenting the package(s)

The FFI challenge
A key challenge in this project is handling the Foreign Function Interface between R and Rust. R stores sparse matrices using the `dgCMatrix` format, which uses a Compressed Sparse Column (CSC) representation with three slots: i (row indices), p (column pointers), and x (values). Since extendr does not provide a native zero-copy wrapper for dgCMatrix, these components must be manually extracted from R and safely wrapped as Rust slices to construct a corresponding sparse matrix using the `faer` library. This requires careful validation of the data and pointers, as incorrect handling can crash the R session. There is a lot more on the FFI problem one could write about and it was best understood after reading the IEEE paper draft shared by the mentor.

1.2.1 Workflow Diagram



1.3 Project Planning

The project will be planned and implemented in different stages. The first stage is understanding the existing problem and the technical challenges we might face. In traditional approaches (such as

1D FFT-based filtering or Savitzky–Golay smoothing), they operate independently on each pixel's spectrum. We plan to exploit the spatial relationships present in hyperspectral data and reduce the time and space complexities.

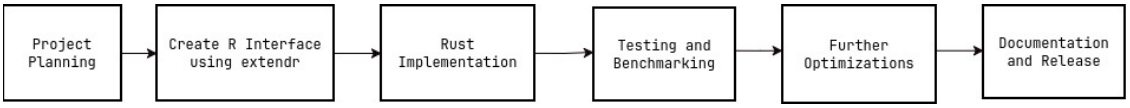
Comparison Table for smoothing using different methods

Method	Time Complexity	Memory Complexity	Estimated Runtime
Dense R (`solve`)	$O(N^3)$	$O(N^2)$	Impractical
Sparse R (`Matrix::solve`)	O(N^1.5)	O(N)	~1 hour
Sparse Rust (CG, `faer`)	O(N √k)	O(N)	< 60 sec

This helps us understand why the Rust implementation is necessary.

I have already covered the key concepts required for this project, including Graph Signal Processing basics, the R–Rust FFI bridge, and the libraries extendr and faer (already used in tests). With this preliminary preparation completed, I will now proceed with the setup and detailed implementation plan for the project.

The following development cycle will be followed



1.3.1 Setup

These are the libraries required for the project, on the R side, Rust side, setup and testing tools. (Addional libraries may be added as per need).

R libraries

1. hyperSpec
2. extendr
3. usethis
4. devtools
5. Matrix
6. testthat

Rust libraries

1. extendr_api
2. faer
3. petgraph
4. thiserror

1.4 Implementation Plan

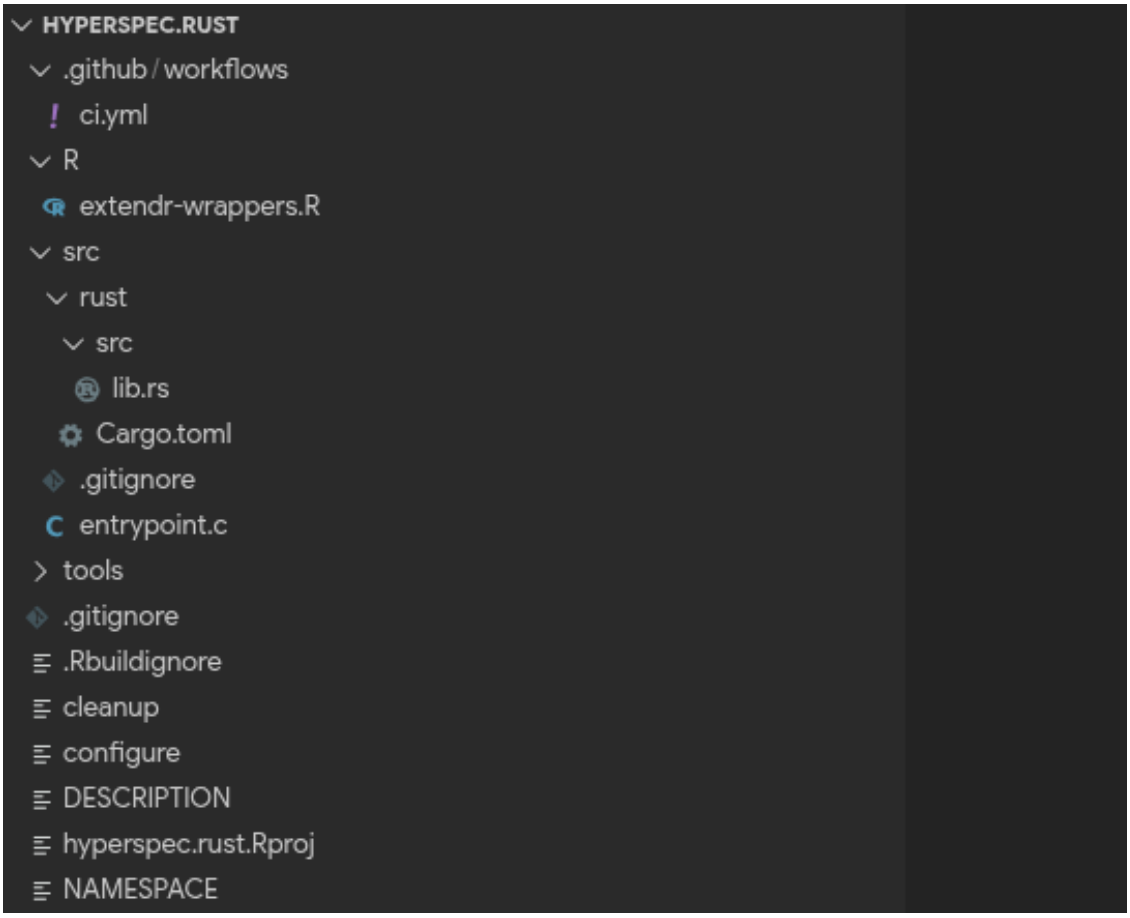
1.4.1 Stage 1: Creation of `hyperSpec.rust` skeleton with CI and basic tests

This stage involves setting up a github repository/ contributing to the existing codebase. I will maintain regular communication with my mentor to ensure that the initial structure of the module

aligns with the project goals and existing architecture of the hyperSpec ecosystem. I will discuss the proposed repository layout, the Rust–R integration approach using extendr, and the setup of continuous integration (CI) for automated builds and tests. Feedback from the mentor will help refine the project skeleton, testing strategy, and coding conventions. We can create the basic skeleton for the project using:

```
library(usethis)
library(rextendr)
usethis::create_package("hyperspec.rust")
setwd("hyperspec.rust")
rextendr::use_extendr()
```

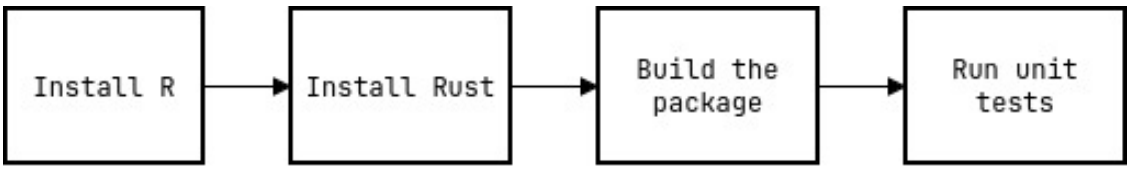
The generated project structure :



We add all our dependencies inside the `hyperspec.rust/src/rust/Cargo.toml` file. Example:

```
[dependencies]
extendr-api = '0.7'
faer = "0.19"
```

Along with that we can also setup some of the CI workflow using github actions using a yaml file in `.github/workflows/` directory.



CI pipeline inside github actions

1.4.2 Stage 2: The dgCMatrix → faer sparse matrix bridge

The extendr package, an R-to-Rust FFI library, does not provide a native wrapper for `dgCMatrix`. This is the FFI problem and the we need to tackle this by constructing a reliable bridge between R's `dgCMatrix` sparse matrix representation and the sparse matrix structures used by the Rust `faer` library. This bridge enables high-performance graph-based computations on hyperspectral datasets while maintaining compatibility with the existing hyperSpec ecosystem.
In R, sparse matrices from the `Matrix` package are stored using the Compressed Sparse Column (CSC) representation through the dgCMatrix S4 class. The CSC structure consists of three arrays: the **value array x**, the **row index array i**, and the **column pointer array p**. These arrays encode all non-zero elements of the matrix and allow efficient traversal of columns. By extracting these arrays and passing them through the extendr interface, the sparse matrix structure can be reconstructed inside Rust using the faer sparse matrix API.

For better understanding the CSC representation, here is a example:

```
Console  Terminal  Background Jobs
R 4.5.2 · ~/stuff/R-stat/hyperspec.rust/
> library(Matrix)
> A <- Matrix(c(0, 0,  0, 2,
+               6, 0, -1, 5,
+               0, 4,  3, 0,
+               0, 0,  5, 0),
+             byrow = TRUE, nrow = 4, sparse = TRUE)
> A
4 x 4 sparse Matrix of class "dgCMatrix"

[1,] . .  . 2
[2,] 6 . -1 5
[3,] . 4  3 .
[4,] . .  5 .
> A@x
[1]  6  4 -1  3  5  2  5
> A@i
[1] 1 2 1 2 3 0 1
> A@p
[1] 0 1 2 5 7
>
```

R-Rust FFI Bridge Implementation
R side: Slot extraction
Since extendr has no native `dgCMatrix` wrapper, we extract the slots: x, i and p in R before calling the Rust function. This is not zero copy as R allocates the slots before passing them. However, each slot is a standard R vector and the transfer time is almost negligible for sparse matrix.

```r
graph_smooth <- function(hy_obj, alpha = 0.5, neighbors = 4L) {

  # Build graph Laplacian (sparse)
  L <- build_pixel_laplacian(hy_obj)

  # Extract dgCMatrix slots
  i_slot <- L@i
  p_slot <- L@p
  x_slot <- L@x

  dims <- dim(L)

  # Call Rust backend
  result <- rust_graph_smooth(
    signal = hy_obj@data$spc,
    i_slot = i_slot,
    p_slot = p_slot,
    x_slot = x_slot,
    nrow = dims[1],
    ncol = dims[2],
    alpha = alpha
  )

  return(result)
}
```

Rust side: Reconstruction and Validation

The Rust function reconstructs the sparse matrix and performs validation before computation. Defensive checks before any pointer arithmetic ensure that malformed input from R cannot cause memory errors.

lib.rs

hyperspec.rust > src > rust > src > lib.rs > rust_graph_smooth

```rust
use extendr_api::prelude::*;

use extendr_api::prelude::*;
use faer::sparse::{SparseColMat, SymbolicSparseColMat};

#[extendr]
fn rust_graph_smooth(
    signal: RMatrix<f64>,
    i_slot: &[i32],
    p_slot: &[i32],
    x_slot: &[f64],
    nrow: i32, ncol: i32,
    alpha: f64,
    ) -> RMatrix<f64> {

    let (nrow, ncol) = (nrow as usize, ncol as usize);

    // Validate before touching memory
    assert_eq!(p_slot.len(), ncol + 1, "p slot length mismatch");
    let nnz = p_slot[ncol] as usize;
    assert_eq!(i_slot.len(), nnz, "i/x slot length mismatch");

    // Convert i32 indices to usize for faer
    let col_ptrs: Vec<usize> = p_slot.iter().map(|&v| v as usize).collect();
    let row_idx: Vec<usize> = i_slot.iter().map(|&v| v as usize).collect();

    // Construct faer sparse matrix
    let sym = SymbolicSparseColMat::new_checked( nrow, ncol, col_ptrs, None, row_idx).expect("invalid CSC structure");
    let laplacian = SparseColMat::new_from_order_and_values(sym, x_slot).expect("build failed");

    // solver function
    solve_per_band(signal, laplacian, alpha)
}
```

Note: R stores slots i and p as `int32`, whereas faer expects `usize` in Rust. This conversion is safe (as $nnz < 2^{31}$ for any realistic sparse matrix), but needs be explicit. Rust's ownership system prevents the most common Rcpp failures:

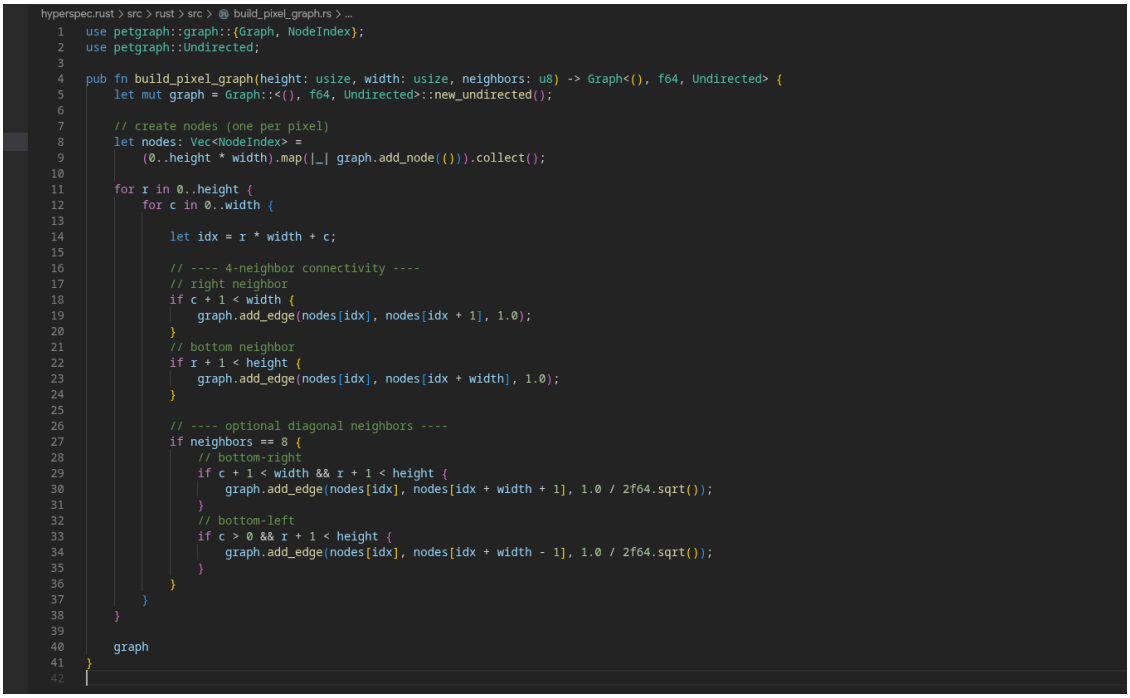
Buffer overflow: Rust's slice indexing panics on out-of-bounds access rather than reading adjacent memory.

Use-after-free: The borrow checker ensures R vectors passed to Rust remain valid for the duration of the function call.

1.4.3 Stage 3: Pixel Neighborhood Graph Construction

In hyperspectral spatial smoothing, the image is modeled as a graph where each pixel is a node and edges connect neighboring pixels. This graph captures the spatial structure of the image and forms the basis for constructing the graph Laplacian matrix, which is later used in the smoothing equation: `(I+ L)x=b`. The Rust library `petgraph` is used to build this graph.

As seen earlier, the pixel neighborhood graph should be configurable by the number of neighbors (e.g, 4 or 8) because different spatial models require different connectivity. This is also consistent with the material where the function accepts a parameter `neighbors` to choose the connectivity.



The types of connectivity we get from this:

neighbors	Connectivity	Description
`4`	4-neighborhood	horizontal + vertical pixels
`8`	8-neighborhood	horizontal + vertical + diagonal

Why connectivity matters?
More neighbours means stronger spatial coupling, and hence smoother results. 4 neighbors mean sharper edges preserved, while 8 neighbors mean smoother results. The configuration may depend based on the type of operation.

Once we have this, the Graph Laplacian can be computed using:

$$L = D - W$$

where W = adjacency matrix and D = degree matrix.
This Laplacian is the core operator used in the smoothing system. Instead of sending a huge adjacency matrix across the R–Rust FFI boundary, Rust reconstructs the graph using only height, width, connectivity. This greatly reduces FFI overhead.

1.4.4 Stage 4: Iterative Solver using CG/BiCGSTAB

Once the Laplacian is built, we need to perform graph-based low-pass filtering, for reducing spatial noise while preserving large structures. Spatial smoothing is formulated as solving the linear system

$$(I + \alpha L)x = b$$

for each wavelength band, where L is the graph Laplacian, b is the original signal, x is the smoothed output, and $\alpha > 0$ controls the smoothing strength.

Since L is sparse, the system can be solved efficiently using iterative methods such as Conjugate Gradient (CG) for symmetric systems or BiCGSTAB for non-symmetric systems. This reduces the computational complexity to $O(N)$ per band, compared to $O(N^3)$ for dense solvers. Both these methods are present in the `faer` library.

Conjugate Gradient Algorithm
If the matrix $I + \alpha L$ is symmetric, then we use this approach. The CG method solves $A_x = b$, where $A = I + \alpha L$. Then the algorithm iteratively improves the solution.

Algorithm steps :-
i. Initialize $x_0 = 0$
ii. compute residual: $r_0 = b - Ax_0$
iii. Update search direction: $p_0 = r_0$
iv. Iterate until convergence

Rust implementation

hyperspec.rust > src > rust > src > solver.rs > solve_cg

```rust
use faer::sparse::SparseColMat;
use faer::solvers::cg::ConjugateGradient;

pub fn solve_cg(
    laplacian: &SparseColMat<usize, f64>,
    b: &[f64],
    alpha: f64,
) -> Vec<f64> {

    let n = b.len();

    // Construct A = I + αL
    let mut A = laplacian.clone();
    for i in 0..n {
        A[(i, i)] += 1.0;     // add identity
        A[(i, i)] += alpha;   // scaled Laplacian
    }

    // Initialize solution vector
    let mut x = vec![0.0; n];

    // Create CG solver
    let mut solver = ConjugateGradient::new();

    solver.solve(&A, &mut x, b);

    x
}
```

BiCGSTAB (Bi-Conjugate Gradient Stabilized) Algorithm
If the matrix $I + \alpha L$ is non-symmetric, then we use this approach.

Algorithm :-
i. Initialize $x_0 = 0$, $r_0 = b - Ax_0$

ii. Choose a shadow residual $\hat{r} = r_0$
iii. Iteration: Compute intermediate scalars and vectors to update the solution:
  update step size
  compute intermediate residual
  compute stabilization factor
iv. Update solution
v. Update residual and direction
vi. Update r_{k+1} and search direction, then repeat until convergence.

Rust implementation

```rust
use faer::prelude::*;
use faer::sparse::SparseColMat;
use faer::iterative_solvers::bicgstab;

pub fn solve_bicgstab(
    laplacian: &SparseColMat<usize, f64>,
    b: &[f64],
    alpha: f64,
) -> Vec<f64> {

    let n = b.len();

    // Build A = I + αL
    let mut A = laplacian.clone();

    for i in 0..n {
        A[(i, i)] += 1.0;
        A[(i, i)] += alpha;
    }

    // Convert vectors to faer format
    let b_vec = Mat::from_fn(n, 1, |i, _| b[i]);

    let mut x = Mat::<f64>::zeros(n, 1);

    // Run BiCGSTAB solver
    bicgstab(
        &A,
        x.as_mut(),
        &b_vec,
        1e-6,   // tolerance
        1000,   // max iterations
    );

    // Convert result back to Vec
    x.col(0).iter().copied().collect()
}
```

1.4.5 Stage 5: Parallel Spectral Processing

Hyperspectral images have many wavelength bands and each solver only solves for one wavelength band. Each band requires solving a separate system: $(I + \alpha L)x_k = b_k$
These solves are completely independent, which means we can run them in parallel using the `Rayon` library in Rust.

```rust
hyperspec.rust > src > rust > src > solve_per_band.rs > ...
use rayon::prelude::*;

fn solve_per_band( signal: &Array2<f64>, laplacian: &SparseColMat<usize, f64>, alpha: f64, ) -> Array2<f64> {

    signal.axis_iter(Axis(1)).into_par_iter().map(|band| solve_cg(laplacian, &band, alpha))
    .collect::<Vec<_>>()
    .into()
}
```

1.4.6 Stage 6: Testing and Benchmarking

Unit Tests
The testing strategy will combine unit tests, integration tests, and performance benchmarks for large datasets. We use the `testthat` package inside the module and add the testfiles in the `hyperspec.rust/tests/testthat/` directory. Then we can run the unit tests automatically using `devtools::test()`. The following 4 unit tests will validate each component of the Rust backend independently:

1. **FFI Bridge Validation:** The first component to test is the conversion from the R dgCMatrix sparse matrix format to Rust sparse structures. The following will be implemented:

i. Correctness in extraction of i, p, and x slots from a dgCMatrix.
ii. Verify: `length(i) == length(x)`, `length(p) == ncol + 1`, column pointers are monotonic, row indices fall within valid bounds.
iii. Proper descriptive error messages returned to R incase of any failure.

2. **Adjacency Graph Construction Tests:** The pixel adjacency graph will be tested to ensure correct connectivity. The following will be implemented:

i. Correct number of nodes for a given grid size.
ii. Correct number of edges depending on the type of connectivity (eg, 4-connectivity or 8-connectivity).
iii. Validate that interior, edge, and corner pixels have correct neighbor counts (eg, a corner pixel has 2 neighbors, edge pixel has 3).

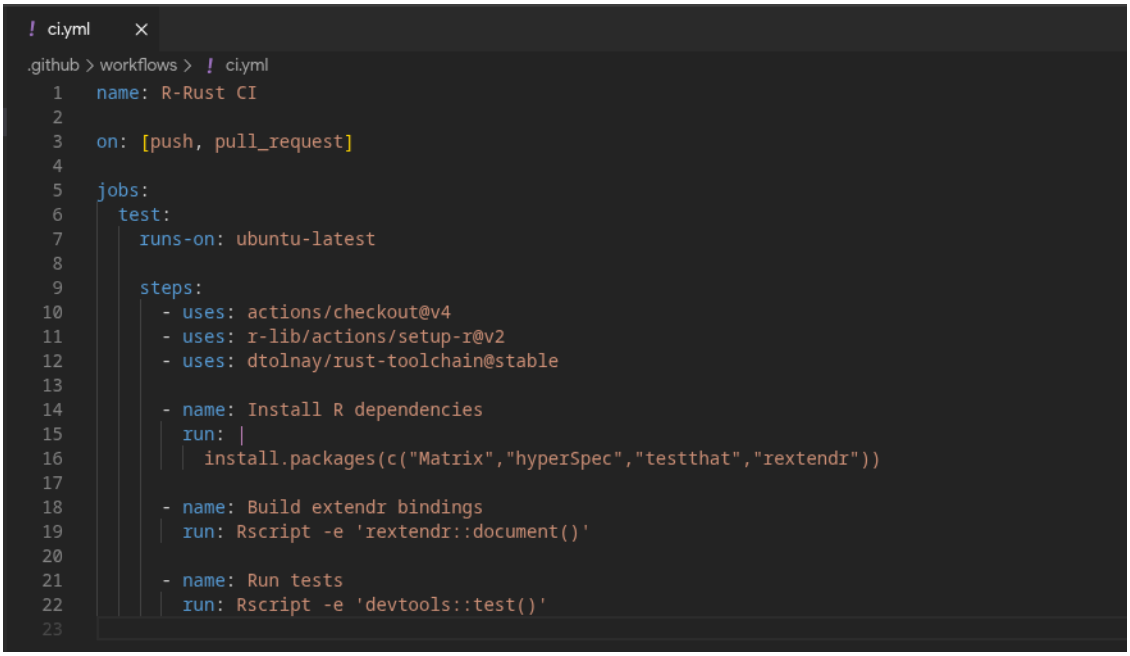
3. **Laplacian Matrix Construction:** The Laplacian construction logic will be validated using small matrices. We first construct the Laplacian and then verify it's properties :

i. diagonal elements equal the node degree.
ii. off-diagonal entries equal -1 for neighbors
iii. row sums equal zero for the Laplacian

4. **Corectness of the iterative solver:** The solver results will be validated for already known results calculated using dense R solver. There might be very small errors and hence we require a tolerence value(almost negligible) for the check.

Integration Test
The Integration tests would verify that the entire processing pipeline: from the R interface to the Rust solver and back to R. These tests will be automated using Continuous Integration (CI) so they run on every commit and pull request. The goal is to ensure:
i. The R interface with graphSmooth(hy, alpha, neighbors) works correctly.
ii. FFI conversion does not crash the R session.
iii. Solver results are correct.
iv. Package builds successfully.

Workflow Structure (inside `.github/workflows/ci.yml`)



Benchmarks
The target benchmark is for a **512 × 512 × 2048** hyperspectral dataset (approximately 4 GB). The 512 × 512 represents the spatial pixel grid and 2048 represents the wavelength dimension. The pure-R implementation has a time complexity of $O(N^3)$ and a space complexity of $O(N^2)$. It would take up **hours to run and about 16GB** of RAM on a laptop (honestly speaking my laptop might just crash). The Rust implementation time complexity is O(N $\sqrt{}$k) and space complexity is $O(N)$, so it would be completed in **less than 1 minute with about 4GB RAM**.

Baseline Comparison: The Rust implementation will be compared against a baseline pure-R approach using sparse matrix operations from the `Matrix` package. In the baseline method, spatial smoothing is performed by solving the linear system $(I + \alpha L)x = b$ using standard R numerical routines such as `Matrix::solve()`.

1.4.7 Documentaion

I will make a developer-friendly documentation for all the Rust functions and the R level method including parameter explainations, examples, performance notes, etc. The `roxygen` and `knitr` packages will be used.

A detailed vignette will demonstrate the complete spatial smoothing workflow on a hyperspectral Raman dataset, explaining the motivation for graph-based filtering and the underlying Laplacian formulation $(I + \alpha L)x = b$. The `rmarkdown` package will be used and the vignette will also include explanations of the Laplacian filtering method, example code, and visualizations of results before and after smoothing.

1.5 Project Obstacles

No project plan is completely free from challenges, and I anticipate a few possible obstacles during the development. One challenge may be ensuring correct and efficient communication between R and Rust through the extendr FFI interface, particularly when handling large hyperspectral

datasets. There might be misunderstandings while understanding certain concepts or errors in implementing code. Additionally, some components may take more time than initially expected. To address these challenges, I plan to set small milestones, maintain regular communication with the mentor and provide progress reports for feedback. I will keep myself enthusiastic and complete the work in time for a successful R GSoC experience.

2 Timeline

2.1 Rough timelime

Date	Event
March 31 - April 30	Pre-GSoC Period
May 1 – May 24	Community Bonding Period
May 25 – July 5	Coding Period 1
July 6 – July 10	Midterm Evaluations
July 11 – August 16	Coding Period 2
August 17 - August 24	Final Week
August 24 - August 31	Endterm Evaluations
August 24 - November 2	Incase of extended coding period
November 2	Final Evaluations (extended)
November 11	Program officially ends

I will try my best to complete the entire project within the standard time period (before August 31), and have prepared the detailed timeline accordingly.

2.2 Detailed Timeline

2.2.1 Pre-GSoC Period (March 31 - April 30)

Week 1
Study the `hyperSpec` repository and read all the R documentation related to it. I will study graph signal processing and deepen my understanding of the existing challenges in order to explore potential optimization approaches. Setup all the neccessary development tools and get familiar with using the different packages.

Week 2
Continue to spend time understanding the codebase and start solving open issues in R. Read documentaation and study the implementations used in various spectroscopic libraries and review the recent developments.

Week 3
Work on improving Rust proficiency. Read the documentation for faer 0.19+ sparse module, focusing on areas of the iterative solver APIs (CG and BiCGSTAB). Read all documentation related to extendr and understand how R S4 objects are handled at the FFI boundary. Read documentation of petgraph and write some Rust code for graph construction and solver experimentation.

Week 4
Strengthen my understanding of R fundamentals. Continue solving issues and contributing. Read documentation and get familiar with the testing libraries.

2.2.2 Community Bonding Period (May 1 - May 24)

Week 1
Establish a clear communication schedule with the mentor, including weekly meetings, progress updates, and discussion channels for technical questions. Finalize the project plan and set milestones for the project. Discuss the development flow for setting up repository, forking, branching, committing, and submitting pull requests.

Week 2
Understand the existing development practices used by the hyperSpec project. Finalize the development environment and tools. Create the initial `hyperspec.rust` skeleton, configure Continuous Integration (CI), and implement a small Rust function callable from R to verify that the Rust–R bridge works correctly. Download all datasets required for testing and benchmarking.

Week 3
Complete stage 1 of the plan: Creation of hyperSpec.rust skeleton with CI and basic tests. Continue reading the resources provided by my mentor. Review the documentation and work on building a prototype for the dgCMatrix $\rightarrow$ faer sparse matrix bridge.

Week 4
Continue reading the resources provided by my mentor. Finalize everything with the mentor and share the report on the prototype.

2.2.3 Coding Period 1 (May 25 - July 5)

Week 1
Complete Stage 1 before the Coding Period. Start working on Stage 2: The R-Rust FFI Bridge. Implement the R side and Rust side code for the bridge. Share the weekly progress report with the mentor.

Week 2
Continue working on Stage 2. Add validation checks and error handling. Verify correctness of the bridge and that the transfer is as intended. Share the weekly progress report with the mentor.

Week 3
Start working on Stage 3: Pixel Neighborhood Graph Construction. Implement the Rust code and verify correctness. Share the weekly progress report with the mentor.

Week 4
Continue working on Stage 3. Implement flexible graph construction configurable by neighbors. Optimize graph construction and validate correctness for larger grids. Share the weekly progress report with the mentor.

Week 5
Start working on Stage 4: Iterative Solver using CG/BiCGSTAB. Implement solver (for symmetric case) in Rust using CG and verify correctness. Share the weekly progress report with the mentor.

Week 6
Implement solver (for non-symmetric case) using BiCGSTAB. Wrap up everything and complete Stage 4. Integrate the solver with the R interface: `graphSmooth(hy, alpha, neighbors)` function callable from R. Share the weekly progress report with the mentor.

2.2.4 Midterm Evaluations (July 6 – July 10)

Write a detailed report on all the work done during the Coding Period. All work will be documented and uploaded.
Midterm Deliverables:
i. hyperspec.rust package skeleton with CI and basic tests.
ii. dgCMatrix → faer sparse matrix bridge.
iii. Pixel adjacency graph construction.
iv. Sparse Laplacian solver in Rust.
v. Functional graphSmooth(hy, alpha, neighbors) R S4 method.

2.2.5 Coding Period 2 (July 11 – August 16)

Week 1
The coding continues. Test the Spatial smoothing working on different datasets and verify that output maintains proper structure. Start working on Stage 5: Parallel Spectral Processing using rayon. Share the weekly progress report with the mentor.

Week 2
Complete work on Stage 5. Test the solver for multiple wavebands parallelly using different datasets. Share the weekly progress report with the mentor.

Week 3
Start working on Stage 6: Testing and Benchmarking. Write unit tests for the different components. Write Integration tests and put everything together in the CI pipeline. Share the weekly progress report with the mentor.

Week 4
Complete stage 6. Work on performance benchmarks using the $512 \times 512 \times 2048$ hyperspectral dataset. Compare pure-R and Rust sparse solver results. Document all benchmark results and performance plots. Share the weekly progress report with the mentor.

Week 5
Work on the last stage. Write detailed documentation and vignette to demonstrate Spatial smoothing of hyperspectral Raman maps using graph filtering. Share the weekly progress report with the mentor.

2.2.6 Final Week (August 17 - August 24)

Wrap up everything. Complete all documentation and any remaining work. Optimize the functions further and refactor code. Test that everything works successfully. Share the weekly progress report with the mentor.

2.2.7 Endterm (August 24 - August 31)

Write a complete project report on all the work done. Write about performance improvements and tests. All work will be uploaded.
Endterm Deliverables:
i. Parallel Spectral Processing.
ii. Unit and Integration tests.
iii.Benchmarks and performance report.
iv. Complete documentation and vignette.

2.3 Contingency Plan

If any setbacks occur due to my health, personal or family reasons, I will communicate with my mentor, explain the difficulty, and seek their guidance on how to adjust the timeline. If technical issues or delays occur during development, I will focus on completing the core components first, particularly the dgCMatrix → faer sparse matrix bridge and the basic sparse Laplacian solver, and then extend the implementation gradually. I have kept around 2 weeks time for managing each stage. If things go south, the schedule can be adjusted by utilizing the extended period.

3 Management of Coding Project

I will track my progress through weekly reports. Additionally, I plan to maintain a blog where I will regularly document updates, which will help ensure that I stay aligned with the planned timeline. During the Community Bonding Period, I will discuss and establish a meeting schedule with my mentor and provide regular progress updates. I will review and incorporate all feedback that is provided. I will commit code several times a week and make sure that all changes are thoroughly tested before each commit.

4 Test

To get prepared for the project, I had done all the three tests(easy, medium and hard) given by the mentor. Through the tests, I learned a lot of things that would be useful for the project like computing pixel adjacency matrix and using it to find the laplacian, creating R package using extendr, writing tests using testthat in R, slot extraction from a R dgCMatrix sparse matrix via extendr and constructing a faer sparse matrix from it. All these tasks gave me experience in using the packages and an insight into the mathematical concepts behind them. The tests were really well designed and it was fun to do them. I made a small writeup for the tasks, on how I solved them along with setup instructions.

4.1 Easy Task

Problem Statement: Install hyperSpec in R. Load the flu or laser dataset. Write a script that constructs a simple pixel adjacency matrix from a 2D spatial grid and computes its graph Laplacian

using base R. Plot the result.
Solution: https://github.com/shinigami-777/hyperSpec-Tasks/tree/main/easy_task

4.2 Medium Task

Problem Statement: Create a minimal R package using extendr. Pass a dense matrix from R to Rust, compute its row sums in Rust using faer, and return the result to R. Include a test.
Solution: https://github.com/shinigami-777/hyperSpec-Tasks/tree/main/medium_task

4.3 Hard Task

Problem Statement: Extract the i, p, and x slots from an R dgCMatrix sparse matrix via extendr. Wrap them as Rust slices and construct a faer sparse matrix. Return the degree vector (row sums) to R using extendr's WritableSlice to avoid an unnecessary allocation, and verify it matches the pure-R result.
Solution: https://github.com/shinigami-777/hyperSpec-Tasks/tree/main/hard_task

5 Anything Else

5.1 Recent Work Related to Project

I had worked on creating a Rust crate with reusable dgCMatrix → faer FFI bridge pattern. Check out https://github.com/shinigami-777/dgcmatrix-faer-bridge. This will be very useful for the implementation of the FFI Bridge in the project as we can directly use the crate/code.

5.2 References

[1] IEEE paper draft on dgCMatrix → faer FFI bridge by Erick Oduniyi
[2] https://letters-photon-processor.web.app/sparse-bridge
[3] https://github.com/eoduniyi/sparse-bridge-visualizations
[4] https://github.com/rstats-gsoc/gsoc2026/wiki/hyperSpec-HPC
[5] https://eoduniyi.github.io/hyperSpec.gsoc2020/